

MLForge Tutorial — Iris-Klassifikator von Null

MLForge

v0.1

Worum es geht

Dieses Tutorial führt dich Schritt für Schritt durch MLForge – vom leeren Workspace bis zum funktionierenden Klassifikator auf dem **Iris**-Datensatz (die “Hello World” der Tabular-ML). Am Ende hast du:

- ein **MLForge-Projekt** mit klarer Ordnerstruktur,
- `iris.csv` als verwalteten Datensatz mit Statistiken und Spaltenauswahl,
- einen kleinen **MLP-Klassifikator** auf dem Canvas, an `iris.csv` gebunden,
- einen erfolgreich durchgelaufenen **Smoke Test** mit echten Daten,
- den generierten PyTorch-Code als `.py`-Datei zum Weiterverwenden.

Aufwand: ca. 15 Minuten. Vorausgesetzt: MLForge läuft (`npm run tauri dev` oder eine installierte `.deb`).

Was ist der Iris-Datensatz? 150 Blütenmessungen von drei *Iris*-Arten (*setosa*, *versicolor*, *virginica*). Vier numerische Features (Kelchblatt-Länge/-Breite, Kronblatt-Länge/-Breite) und eine kategoriale Zielspalte (`species`). Klassisch, klein, linear trennbar – ideal zum Aufwärmen.

1 – Projekt anlegen

Beim Start zeigt MLForge ein **Welcome-Panel**. Klicke auf “*Projekt öffnen / Ordner wählen...*” und wähle (oder erstelle) einen Ordner für dieses Projekt – z.B. `~/projects/mlforge-iris-tutorial`.

- Ist der Ordner **leer**, erscheint “*Projekt initialisieren*”.
- Hat er bereits lose `.mlforge`-Dateien, kommt “*In Projekt konvertieren*”.

Klick drauf, fülle das Formular aus:

Feld	Beispiel
Name	<code>iris-classifier</code>
Description	Erstes End-to-End-Tutorial mit MLForge.
Goal	3-Klassen-Klassifikator für Iris auf Kron-/Kelchblatt-Maßen.

Goal ist nicht nur Doku – **Claude liest dieses Feld bei jedem Chat-Turn**, versteht so den Kontext und schlägt passende Layer vor.

Nach dem Klick auf “*Projekt anlegen*” hast du folgende Struktur:

```
iris-classifier/
|-- mlforge.project.json    <- Metadaten
|-- models/                <- .mlforge-Dateien
|-- datasets/              <- Datensätze
|-- notes/                 <- Markdown-Notizen (Claude darf hier schreiben)
`-- experiments/           <- smoke-results.jsonl etc.
```

Die Toolbar oben zeigt jetzt **MLForge * iris-classifier**.

2 – Iris-Datensatz besorgen

Falls noch nicht da, kannst du `iris.csv` aus mehreren Quellen ziehen. Variante mit `curl` direkt in den Workspace:

```
cd ~/projects/mlforge-iris-tutorial/datasets
curl -s https://gist.githubusercontent.com/curran/a08a1080b88344b0c8a7/raw/iris.csv \
  > iris.csv
head -3 iris.csv
```

Die CSV hat fünf Spalten: vier Float-Features und eine String-Klasse.

Im **Datasets-Tab** der linken Sidebar (neben *Files*) das [reload]-Icon drücken – `iris.csv` taucht jetzt als Eintrag auf. Klick öffnet das Detail-Modal.

3 – Datensatz erkunden

Das Modal hat drei Tabs:

Overview – `head()` der ersten Zeilen, dtypes je Spalte, ein “*Use [1, 5] as input*”-Button (setzt die Input-Shape direkt am aktiven Input-Node).

Stats – pro Spalte: mu, sigma, min/max, Anzahl unique/missing, ein kleines Histogramm. Unten eine Korrelationsmatrix (numerische Spalten):

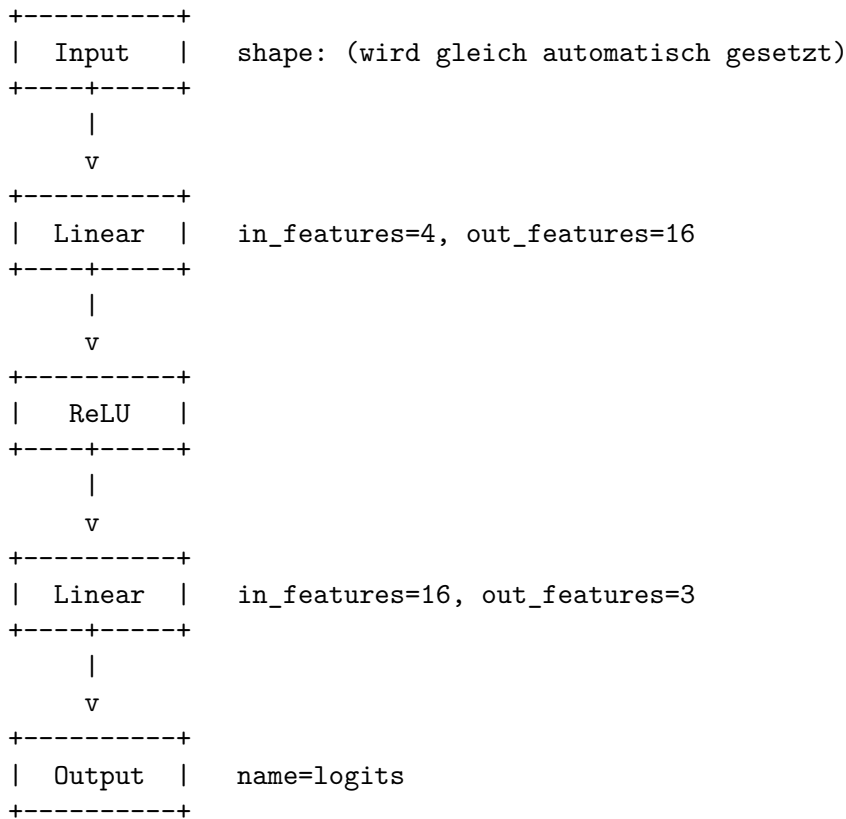
	sepal_length	sepal_width	petal_length	petal_width
sepal_length	1.00	-0.11	0.87	0.82
sepal_width	-0.11	1.00	-0.42	-0.36
petal_length	0.87	-0.42	1.00	0.96
petal_width	0.82	-0.36	0.96	1.00

Petal-Länge und -Breite korrelieren stark (0.96) und beide stark mit Kelch-Länge – ein gutes Zeichen dafür, dass das Problem leicht trennbar ist.

Smoke test – kommt gleich (Schritt 6).

4 – Modell auf dem Canvas bauen

Aus der **Palette** links die folgenden Layer per Drag-and-Drop auf den Canvas ziehen und mit Pfeilen verbinden:



Wichtig: für tabulare Daten **immer Linear** wählen, nicht Conv2d. Das setzt MLForge auch automatisch um – wenn du versehentlich einen Conv-Block drauf hast und ein 2D-Tabular-Input dahinter, zeigt der Inspector einen amber Button *“swap -> mit Linear ersetzen (in_features=4)”* zum Reparieren in einem Klick.

Im **Code-Preview** unten siehst du parallel den generierten PyTorch-Code wachsen – `forward(self, x): ....` Das ist der echte Code, den der Smoke Test und ein Training laufen lassen würden.

5 – Dataset an den Input binden

Den **Input-Node** anklicken. Im **Inspector** rechts erscheinen die Felder:

Feld	Wert nach Auswahl
name	x (oder x1 bei mehreren Inputs)
shape	wird automatisch gesetzt
dataset	Dropdown -> datasets/iris.csv
features	Checkbox-Liste (s.u.)
target	species

Beim Auswählen von `iris.csv` passiert mehrere Sachen gleichzeitig:

- `shape` springt auf `[1, 4]` (alle numerischen Spalten).
- Unter dem Dropdown erscheint eine Mini-Card: *“Dataset-Shape: [1, 4] OK shapes match”*.
- Die `features`-Liste wird vorausgefüllt mit allen vier numerischen Spalten.
- `target` ist noch leer – setze es auf `species`, damit du weißt was gelernt werden soll (für den Smoke Test irrelevant, aber gut zu wissen).

Wenn du z.B. nur Kronblatt-Maße nutzen willst, **haken** in der `features`-Liste nur `petal_length` und `petal_width` – `shape` springt sofort auf `[1, 2]`, und der erste Linear-Layer zeigt im Inspector den roten Hint *“in_features has 2”* mit einem ein-Klick-Fix. Damit ist dein Modell von Drei-Sekunden-zu-Vier-Sekunden-Coding-Stil weg.

6 – Smoke Test laufen lassen

Im Dataset-Modal den **Smoke test**-Tab öffnen. Du siehst:

Schickt einen Sample-Batch aus dem Datensatz durch das aktuelle Modell.

```
input shape vom Graph: [1, 4]
[ Run smoke test ]
```

Klick auf den Button. Innerhalb von einer halben Sekunde steht:

```
OK forward pass succeeded
input:  [1, 4]
output: [1, 3]
params: 131
timings: sample 80ms * forward 2ms
iris.csv: used first 1 rows x 4 chosen cols
         (sepal_length, sepal_width, petal_length, petal_width)
```

Was bedeutet das?

- **input [1, 4]** – eine Zeile aus iris.csv, vier Features, an `forward(x)`.
- **output [1, 3]** – drei Logits, einer je Klasse – passt zur Ziel-Aufgabe.
- **131 params** – sehr klein, läuft auf jedem Laptop in Millisekunden.
- **forward 2ms** – das Modell hat tatsächlich gerechnet, kein Mocking.

Der Lauf wird in `experiments/smoke-results.jsonl` angehängt; in folgenden Sessions zeigt der Smoke-Tab unten eine **History** der letzten Runs.

Was wenn's fehlschlägt? Der Inspector zeigt eine farbige Card mit Stage (*Sample, Compile, Construct, Forward*), dem PyTorch-Fehlertext, der erwarteten vs. tatsächlichen Shape, und einem Hinweis je nach Stage. Ist die Architektur das Problem, gibt's einen Ein-Klick-Button zum Reparieren – z.B. *"Input-Shape auf [1, 4] setzen"* oder *"mit Linear ersetzen"*.

7 – Mit Claude iterieren

Im Chat-Panel rechts unten:

Du: Mach mir aus diesem Modell einen ordentlichen Tabular-Klassifikator mit BatchNorm und Dropout. Drei Klassen.

Claude kennt jetzt durch den Projekt-Kontext bereits:

- **name:** iris-classifier
- **goal:** 3-Klassen-Klassifikator für Iris auf Kron-/Kelchblatt-Maßen
- **active_dataset:** `datasets/iris.csv` (inkl. Spalten + Stats)
- **active_model:** dein offenes `.mlforge`

und nutzt seine MCP-Tools, um den Graph **direkt zu bearbeiten**: er fügt `BatchNorm1d`, `Dropout`, ggf. einen zweiten Hidden-Layer ein, prüft die Shapes. Im Chat-Verlauf siehst du die Tool-Calls inline.

Wenn du z.B. willst, dass Claude eine Entscheidung dauerhaft festhält:

Du: Schreib in `notes/decisions.md`, warum wir BatchNorm vor ReLU setzen.

Claude ruft `append_note` auf und du findest beim nächsten Start des Projekts eine Markdown-Notiz, die er beim nächsten Chat-Turn auch wieder lesen kann. So entsteht **Session-übergreifende Kontinuität**.

8 – Code exportieren

Im File-Explorer auf die `.mlforge`-Datei rechtsklicken -> *“View generated PyTorch...”*. Das öffnet ein Monaco-Editor-Modal mit dem vollständigen Modell. Ein typisches Ergebnis:

```
import torch
import torch.nn as nn

class Model(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear_1 = nn.Linear(in_features=4, out_features=16, bias=True)
        self.re_lu_1 = nn.ReLU(inplace=False)
        self.linear_2 = nn.Linear(in_features=16, out_features=3, bias=True)

    def forward(self, x):
        linear_1 = self.linear_1(x)
        re_lu_1 = self.re_lu_1(linear_1)
        linear_2 = self.linear_2(re_lu_1)
        return linear_2

if __name__ == "__main__":
    model = Model()
    n_params = sum(p.numel() for p in model.parameters())
    print(f"Parameters: {n_params:,}")
    x = torch.zeros((1, 4))
    out = model(x)
    print(f"Output shape: {tuple(out.shape)}")
```

Eine `.py`-Datei mit demselben Inhalt liegt automatisch neben der `.mlforge` in `models/`. Du kannst sie 1:1 in dein Training-Notebook übernehmen – z.B. so:

```
import pandas as pd, torch, torch.nn as nn, torch.nn.functional as F
from models.iris_classifier import Model

df = pd.read_csv("datasets/iris.csv")
X = torch.tensor(df[["sepal_length", "sepal_width",
                    "petal_length", "petal_width"]].values, dtype=torch.float32)
y = torch.tensor(pd.Categorical(df["species"]).codes, dtype=torch.long)

model = Model()
opt = torch.optim.Adam(model.parameters(), lr=1e-2)
for epoch in range(200):
    opt.zero_grad()
    loss = F.cross_entropy(model(X), y)
    loss.backward(); opt.step()
print("acc:", (model(X).argmax(1) == y).float().mean().item())
```

Auf einem üblichen Laptop landet die Accuracy nach 200 Epochs bei ~0.97.

9 – Cheatsheet

Aktion	Wo
Projekt initialisieren	Welcome-Screen
Datensätze einsehen	Sidebar -> <i>Datasets</i> -Tab
Spalten als Features wählen	Inspector -> Input-Node -> features
Dataset an Input binden	Inspector -> dataset -Dropdown
Smoke Test	Dataset-Modal -> <i>Smoke test</i> -Tab
Rang-Mismatch reparieren	Inspector -> amber “ <i>swap</i> -> <i>ersetzen</i> ”-Button
Generierten Code anschauen	File-Explorer -> Rechts-Klick auf .mlforge
Claude eine Entscheidung notieren lassen	Chat: “ <i>schreib in notes/X.md ...</i> ”

10 – Weiter

- Spiel mit der Architektur: tieferes MLP, Residual-Verbindungen (Add-Layer), zwei Inputs (z.B. Kelchblatt vs. Kronblatt getrennt, dann Concat).
- Probier ein zweites Dataset im selben Workspace – **datasets/** darf beliebig viele Einträge haben.
- Frag Claude nach Loss-/Optimizer-Empfehlungen für deinen konkreten Anwendungsfall.

Viel Spaß. Wenn dir was fehlt: das Issue-Template ist **gh issue create** im Repo, oder schreib es in **notes/wishlist.md** – Claude wird’s beim nächsten Mal lesen.